BreakBite

Smart Food, Smart Time

Complete Project Documentation and Rebuild Guide

Team BITE

Gono Bishwabidyalay - CSE Software Development Contest 2026
Category 1: Semester 1st to 4th
Document date: 20 October 2026
Live site: https://raito-del.github.io/BreakBite/
Repository: github.com/raito-del/BreakBite

Confidential team document. Contains no passwords or private keys.

Contents

1. Team and roles

2. Project overview and idea

3. Thinking process and design decisions

4. Features (complete list)

5. Architecture and technology

6. File inventory (every file)

7. Data model (Firestore)

8. How the main features work

9. Design system and theme

10. Security status and known limitations

11. Rebuild guide (step by step)

12. Work log and timeline

13. Future work

Appendix A. Key source code

1. Team and roles

Team name: BITE. Project name: BreakBite (Break Bite). The team builds one connected canteen web app. Each member owns a set of files, so work does not clash on GitHub.

Name	Student ID	Responsibility	Owned files
Nahar Mim (Team Lead)	26251037	Lead, documentation, home page	index.html, README.md
Sadnan Ahmed	26251038	Student ordering pages	student.html (ordering flow)
Wasika Jahan	26251026	UI design and styling	style.css, visual design
Mirza Muin Labbaik	26251021	Staff page and database setup	staff.html, firebase.js
Habiba Jannat Taiyaba	26251011	Break Planner and testing	planner.html

Shared files (style.css, firebase.js) are changed carefully: one person edits, others pull first. The team uses GitHub Desktop: Fetch origin, Pull origin, work, Commit, Push.

2. Project overview and idea

University students usually have a short break between classes, often 15 to 45 minutes, and limited money. The canteen has long queues, so students waste part of the break waiting, and often cannot decide what to eat in time. BreakBite solves this with three ideas:

- Pre-order: choose food from a live menu, get a token, and collect it when it is ready.
- Rush radar: show how busy the canteen is, so students know when to come.
- Break Planner: enter break time and money, and get a suggested set of items that fits both.

The staff side runs the canteen from one page: live order board, menu and price control, food waste discounts, planner settings, and an end-of-day report.

Target user: a first-year university student who has 30 minutes and some money, and wants a fast, clear decision.

3. Thinking process and design decisions

3.1 Why plain HTML, CSS and JavaScript?

The team is at an early university level. A build tool or framework adds setup steps that can break for teammates. Plain files open directly in Live Server or GitHub Pages, and every member can read them. Firebase is added through its CDN modules, so there is no npm step.

3.2 Why Firebase?

Orders must be shared between many devices in real time: a student's phone and the staff tablet. Firestore gives live updates (onSnapshot) on the free Spark plan. Firebase Authentication gives staff a real login without storing passwords in code.

3.3 Why one stylesheet?

Team members wanted different themes. All colours and shapes are CSS variables in one style.css file. Changing four colours re-themes every page, so no HTML needs editing.

3.4 Why a shared order counter?

Token numbers must be unique even when two students press the button at the same moment. A Firestore transaction reads counters/orders, adds one, and writes the new order in one safe step.

3.5 Why a daily report and reset?

A canteen runs one day at a time. Keeping every order forever makes the board messy and tokens grow forever. End-of-day saves a summary first, then clears orders and restarts tokens at 1. The report is saved before anything is deleted, so no sales data is lost if the process fails.

3.6 Why a knapsack-style planner?

The planner solves a small version of the 0/1 and unbounded knapsack problem: items have time and price, the budget has time and money limits. A dynamic programming table gives the best answer quickly for small menus, without a server.

3.7 Trade-offs we accepted (honest view)

- Firestore rules are open for the contest demo. This is not secure for real money or private data (see section 10).
- The planner maximises item count, so it can repeat one cheap item many times. A variety rule is planned.
- Token lookup lets anyone read an order by token number. The data is low sensitivity, but it is public.
- The site has no build step, so code is simple but not split into modules.

4. Features (complete list)

Area	Feature	Where	Status
Student	Live menu with prices, discounts and availability	student.html	Done
Student	+ / - quantity steppers and sticky cart bar	student.html	Done
Student	Place pre-order with a transactional token	student.html	Done
Student	Multiple active orders kept per browser	student.html	Done
Student	Token status steps (Received, Preparing, Ready)	student.html	Done
Student	Token lookup by number	student.html	Done
Student	Rush radar (orders waiting)	student.html	Done
Student	Break Planner with time and money input	planner.html	Done
Staff	Email and password login (Firebase Auth)	staff.html	Done
Staff	Live order board with advance buttons	staff.html	Done
Staff	Completed list (latest 10)	staff.html	Done
Staff	Menu: add, edit, sold out, delete	staff.html	Done
Staff	Food Waste Saver discounts (5 to 90 percent)	staff.html	Done
Staff	Break Planner settings (min, max, step, default eat time)	staff.html	Done
Staff	End of day: report, delete orders, reset tokens	staff.html	Done
Staff	Past daily reports (last 7)	staff.html	Done
Home	Student and Staff role switch	index.html	Done
Design	Mobile-friendly layout, animations, theme variables	style.css	Done
Future	Green Token and student-side Food Waste Saver	-	Planned
Future	Planner variety rule, stricter security rules	-	Planned

5. Architecture and technology

Layer	Technology	Notes
Pages	HTML5, CSS3, vanilla JavaScript (ES modules)	No framework, no build step
Styling	One file: style.css with :root variables	Mint theme, Spice Kitchen proposed
Database	Cloud Firestore (asia-south1, Mumbai)	Live listeners with onSnapshot
Login	Firebase Authentication, Email/Password	Staff accounts created in console
SDK	Firebase Web SDK 10.12.2 from gstatic CDN	Version pinned on purpose
Hosting	GitHub Pages from branch main, root folder	https://raito-del.github.io/BreakBite/
Editor	VS Code with Live Server	Test pages on localhost
Version control	Git and GitHub Desktop	.gitattributes sets text=auto (LF in repo)

Flow: a student page reads menu and writes orders. The staff page reads orders and changes status. Both pages reconnect to Firestore with the same firebase.js. Firestore pushes updates to every open page, so no refresh is needed.

6. File inventory (every file)

Project root folder. All files sit in the same folder, because pages link to each other with relative paths (for example href='student.html').

File	Purpose	Key contents
index.html	Home page	Header tabs, Student and Staff role links, hero text, two cards (Order, Break Planner)
student.html	Ordering page	Rush radar, menu list, cart bar, place order transaction, token tracking, your orders, token lookup
planner.html	Break Planner	Loads settings/planner and menu once, best-combination search, result list
staff.html	Staff dashboard	Login screen, Orders board, Completed list, End of day, reports, Menu, Food Waste Saver, Settings
style.css	Shared stylesheet	Theme variables, buttons, cards, tickets, steps, tables, login, responsive rules, animations, doodle background
firebase.js	Firebase connection	Config object, initializeApp, exports db (Firestore) and auth (Authentication)
README.md	Project information	Name, description, features, team table, folder structure, workflow, dates
.gitattributes	Line endings	Single rule: * text=auto
yellow-preview.html	Theme preview (not in site)	Early theme experiment with colour picker and presets
spice-kitchen-preview.html	Static design preview (not in site)	Home, student, staff login and staff board in Spice Kitchen colours
spice-kitchen-animated-preview.html	Animated preview (not in site)	Working demo of button, cart, token and step animations
BreakBite_Break_Planner_Presentation.pdf	Showcase slides (not in site)	10-slide story presentation in Bengali

6.1 What lives inside each page (quick map)

- index.html: no database. Only links and role icons. Staff link opens staff.html, which asks for Firebase login.
- student.html: listeners on menu (ordered by order field), orders with status Received or Preparing (rush radar), and each saved order id (order status). Writes new orders and the counter.
- planner.html: one read of settings/planner and one read of menu; no live listener. Pure client-side calculation.
- staff.html: login gate, listeners on orders (latest 50) and menu, writes status changes, menu edits, discounts, settings, daily reports and counter reset.
- style.css: no JavaScript. Shared by all pages through <link rel='stylesheet' href='style.css'>.
- firebase.js: no UI. Imported by every page that uses the database or login.

7. Data model (Firestore)

Collection / document	Fields	Written by	Read by
menu/{id}	name, price (Tk), minutes (eat time), available (bool), discount (0-90), order (sort key)	Staff	Student, Planner, Staff
orders/{id}	token, items [{name, qty, price}], count, total, status, createdAt	Student (create), Staff (status)	Student (own), Staff, Rush radar

Collection / document	Fields	Written by	Read by
counters/orders	last (number)	Student (transaction), Staff (reset)	Student (transaction)
settings/planner	minBreak, maxBreak, step, defaultEat	Staff	Planner, Staff
daily_reports/{id}	date, totalOrders, totalSales, items {name: qty}, activeAtClose, createdAt	Staff (End of day)	Staff (last 7)

Status values used everywhere: Received, Preparing, Ready, Done. The order board shows the first three. Done orders leave the board and appear in Completed.

8. How the main features work

8.1 Placing an order (student.html)

- Student changes quantities; each + or - updates a local object qty. The cart bar shows the count and total.
- On Place pre-order, a Firestore transaction reads counters/orders, sets last = last + 1, writes the order with status Received.
- The new order id is saved in localStorage (key breakbite_orders), so the student can see it after a refresh.
- The student page listens to that order document and shows steps and a message. When status becomes Done, the order leaves the list.

8.2 Staff order flow (staff.html)

- Orders listener loads the latest 50 orders by createdAt. Active orders fill three columns.
- Each button moves an order one step: Received to Preparing, Preparing to Ready, Ready to Done.
- Queue and waiting time: queue = count of Received and Preparing; estimated wait = 3 + queue x 2 minutes.

8.3 Rush radar

The student page listens to orders with status Received or Preparing. Count below 5 shows Low rush, 5 to 9 shows Medium rush, 10 or more shows High rush.

8.4 Break Planner (planner.html)

The planner reads settings and menu once. It checks that the break time is between minBreak and maxBreak. It then runs a dynamic programming table over time (minutes) and spend (Tk). For each state it stores the largest item count and the item used. It finally picks the state with the most items, then the largest spend, and walks back to list the items. Discounted prices are used. Items with no minutes use defaultEat. Appendix A shows the core code.

8.5 End of day (staff.html)

- Staff presses End of day. Two confirmation questions appear. The second one warns that it cannot be undone.
- The app reads all orders, builds the summary (count, total sales, quantity per item). Nothing is deleted yet.
- It writes one daily_reports document. If this write fails, the process stops and no order is deleted.
- Orders are deleted in batches of up to 400. Then counters/orders is set to last = 0, so the next token is 1.
- Past reports (latest 7) appear at the bottom of the Orders tab.

8.6 Login and logout

Staff enter email and password on the login screen. Firebase checks them. The app shows the dashboard only when onAuthStateChanged reports a signed-in user. Logout calls signOut. No password is stored in code or in the browser.

8.7 Discounts (Food Waste Saver)

Staff choose a food and a percent from 5 to 90. The menu document gets discount = percent. Student, planner and staff pages all show the discounted price. Stop discount sets discount back to 0.

9. Design system and theme

All colours are variables in :root of style.css. A theme change means editing only these values.

Variable	Mint theme (live now)	Spice Kitchen (proposed)	Used for
--bg	#d9f6e4	#fff3dc	Page background
--ink	#0d3326	#3b2314	Text, borders, shadows
--accent	#1f9d6b	#d9482b	Hover, highlights, numbers
--card	#ffffff	#ffffff	Cards and header
--danger	#c0392b	#b02a1a	Delete, errors
--turmeric	(not used)	#f2b33d	Tabs, badges, cart button
--t-yellow / --t-blue / --t-mint	#ffe7a0 / #d4e6f7 / #c5ecd5	#ffe08a / #ffd0a8 / #cfeab0	Received / Preparing / Ready

9.1 Animations

- Button hover: small wiggle and lift. Press: pushes down.
- Menu row: lifts on hover with a soft shadow.
- Cart bar: slides up on page open.
- Current order step: soft red glow.
- Ready token: gentle pulse. Works in Chrome and Edge through :has().
- Spinner on a button when it has the class loading.
- All animation is switched off for users who set reduced motion.

9.2 Food doodle background

A small inline SVG pattern (burger, pizza, tea cup, samosa, fries, drumstick) sits behind the page at 10 percent opacity. It needs no image file, so the page stays fast.

10. Security status and known limitations

Topic	Current state	Risk	Planned fix
Firestore rules	Open for read and write until 1 Dec 2026	Anyone can edit menu or orders	Rules that require auth for menu, settings, reports, counters
Staff login	Firebase Email/Password	Accounts must be added by admin	Keep Email/Password only; disable sign-up
Staff gate on home page	Staff icon opens login page	Low	Keep; login page is the real gate

Topic	Current state	Risk	Planned fix
Token lookup	Anyone can read an order by token	Low (only status and items)	Accept, or restrict with a per-order code
Planner variety	Can repeat one item many times	Quality, not security	Cap repeats per item
Token numbers	Keep counting until End of day	Low	Daily reset is manual, by design
Web config in code	apiKey is public by design	Low	Secure with rules, not by hiding the key

Recommended rules for the final showcase (write the target, then publish): students may read menu and settings, create orders and read their own order; only signed-in staff may update orders, change menu and settings, write daily reports, and reset counters. Appendix A includes the target rules.

11. Rebuild guide (step by step)

Follow these steps to recreate BreakBite from nothing. Replace the placeholder values with your own Firebase settings. Never write a real password into code or into this document.

1. Create a GitHub repository, for example BreakBite. Add a README.md.
2. Create the files listed in section 6 in one folder. Copy the code from Appendix A and from the team repository.
3. Open console.firebase.google.com. Create a project. Turn Google Analytics off.
4. Add a Web app (the </> icon). Copy the six values from firebaseConfig into firebase.js.
5. Build > Firestore Database > Create database. Choose Standard, location asia-south1 (Mumbai), start in test mode.
6. Firestore > Rules: paste the rules from Appendix A (temporary open rules for demo) and Publish.
7. Build > Authentication > Get started > Sign-in method > Email/Password > Enable > Save.
8. Authentication > Users > Add user: add each staff email and a password of at least 6 characters.
9. Authentication > Settings > Authorized domains: add raito-del.github.io (or your GitHub Pages domain).
10. In Firestore, create collection counters, document orders with field last = 0 (number).
11. Open the repo on GitHub > Settings > Pages > Source: Deploy from a branch > main > / (root) > Save.
12. Wait 1 to 2 minutes. Open https://<your-user>.github.io/BreakBite/ and test every page.
13. Open staff.html, log in, add menu items with name, price and eat time, then save planner settings.
14. Place a test order from student.html. Advance it on staff.html. Check the status on student.html.
15. Press End of day once to test the report, then check Past daily reports.

12. Work log and timeline

Date (2026)	Milestone
18 October	Repository created, first pages, Firebase guide written for the team
19 October	Firebase project and Firestore set up; firebase.js; first student and staff pages connected
19 to 20 October	Break Planner built; staff login with Firebase Auth; style.css unified; mobile fixes
20 October	GitHub Pages live; home page with Student and Staff icons; token lookup; End of day and reports; theme previews and animations; presentation PDF
12 November	Final project submission (deadline)
16 November	Showcase and poster

13. Future work

- Tighten Firestore rules as in Appendix A, and test with a second staff account.
- Add a variety rule to the Break Planner (for example, one item at most twice).
- Green Token: a student-side feature for reusable containers.
- Student-side Food Waste Saver: show discount items on the order page with a clear badge.
- Apply the Spice Kitchen theme after review, by changing only the colour variables in style.css.
- Optional: daily automatic reset with a scheduled Cloud Function (needs the Blaze plan).

Appendix A. Key source code

The full, current source of every page is in the GitHub repository. The excerpts below are the parts that carry the important logic. Placeholder values replace the real Firebase keys.

A.1 firebase.js (connection file, placeholders only)

```
// firebase.js: the only place where the Firebase settings live.
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-app.js";
import { getFirestore } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";
import { getAuth } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.firebasestorage.app",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID"
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);
```

A.2 Placing an order with a safe token (student.html)

```
const counterRef = doc(db, "counters", "orders");
const orderRef = doc(collection(db, "orders"));
await runTransaction(db, async function (tx) {
  const cs = await tx.get(counterRef);
  const next = (cs.exists() ? cs.data().last : 0) + 1;   // next token number
  tx.set(counterRef, { last: next });
  tx.set(orderRef, {
    token: next, items: items, count: count, total: total,
    status: "Received", createdAt: serverTimestamp()
  });
});
addId(orderRef.id);   // remember this order in this browser
```

A.3 Planner: best combination (planner.html)

```
function bestCombo(minutes, money) {
  const cap = Math.min(money, 1500);
  const W = cap + 1;
  const size = (minutes + 1) * W;
  const count = new Int16Array(size).fill(-1);  // -1 means not reachable
  const from = new Int16Array(size).fill(-1);   // which food was added last
  count[0] = 0;
  for (let i = 0; i < foods.length; i++) {      // unbounded knapsack
    const ft = foods[i].minutes, fp = foods[i].price;
    if (ft > minutes || fp > cap) continue;
    for (let t = ft; t <= minutes; t++) {
      for (let m = fp; m <= cap; m++) {
        const prev = (t - ft) * W + (m - fp);
        if (count[prev] < 0) continue;
        const idx = t * W + m;
        if (count[prev] + 1 > count[idx]) { count[idx] = count[prev] + 1; from[idx] = i; }
      }
    }
  }
  // pick the state with most items, then the bigger spend
  let bestIdx = 0, bestCount = 0, bestSpend = 0;
  for (let t = 0; t <= minutes; t++) for (let m = 0; m <= cap; m++) {
    const c = count[t * W + m];
    if (c > bestCount || (c === bestCount && c > 0 && m > bestSpend)) {
      bestCount = c; bestSpend = m; bestIdx = t * W + m;
    }
  }
  // walk back to list the chosen items
  const picked = []; let idx = bestIdx, guard = 0;
  while (idx > 0 && from[idx] >= 0 && guard++ < 200) {
    const f = foods[from[idx]]; picked.push(f);
    const t = Math.floor(idx / W), m = idx % W;
    idx = (t - f.minutes) * W + (m - f.price);
  }
  return picked;
}
```

A.4 End of day: report first, delete second (staff.html)

```
await addDoc(collection(db, "daily_reports"), {        // 1. save report first
  date: todayKey(), totalOrders: all.size, totalSales: totalSales,
  items: itemCount, activeAtClose: activeCount, createdAt: serverTimestamp()
});
const docs = all.docs;                                  // 2. delete in batches of 400
for (let i = 0; i < docs.length; i += 400) {
  const batch = writeBatch(db);
  docs.slice(i, i + 400).forEach(function (d) { batch.delete(d.ref); });
  await batch.commit();
}
await setDoc(doc(db, "counters", "orders"), { last: 0 });   // 3. restart tokens
```

A.5 Current rules (open, demo only)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.time < timestamp.date(2026, 12, 1);
    }
  }
}
```

A.6 Target rules for the showcase (recommended)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isStaff() { return request.auth != null; }

    match /menu/{id}      { allow read: if true;  allow write: if isStaff(); }
    match /settings/{id}  { allow read: if true;  allow write: if isStaff(); }
    match /counters/{id}  { allow read: if true;  allow write: if true; }
    match /orders/{id} {
      allow read: if true;
      allow create: if true;
      allow update, delete: if isStaff();
    }
    match /daily_reports/{id} { allow read: if isStaff(); allow create: if isStaff(); }
  }
}
```

Note: counters must allow create and update from students for token numbers to work. Test every page after changing rules.

Appendix B. Complete source of every file

This appendix contains the full text of each project file as it is in the repository. The Firebase settings in firebase.js are replaced by placeholders on purpose, so this document is safe to share. Put your own values from the Firebase console into your local firebase.js.

Order	File	Lines (approx.)
B.1	index.html	home page
B.2	firebase.js	Firebase connection (placeholders)
B.3	student.html	ordering page
B.4	planner.html	break planner
B.5	staff.html	staff dashboard
B.6	style.css	shared stylesheet, Spice Kitchen theme
B.7	README.md	project information
B.8	.gitattributes	line endings

B.1 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite</title>
  <link rel="stylesheet" href="style.css">
  <style>
    .role-switch { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 20px; }
    .role-btn {
      display: flex; align-items: center; gap: 10px;
      background: var(--card); color: var(--ink);
      border: 2px solid var(--ink); border-radius: 12px;
      padding: 12px 18px; box-shadow: var(--shadow-small);
      font-weight: 800; font-size: 16px; cursor: pointer; text-decoration: none;
    }
    .role-btn .ico {
      display: inline-grid; place-items: center;
      width: 34px; height: 34px; border-radius: 9px;
      background: var(--ink); color: var(--card); font-size: 18px;
    }
    .role-btn.on { background: var(--t-mint); }
    .role-btn:hover { background: var(--accent); color: #fff; }
    .role-btn:hover .ico { background: #fff; color: var(--ink); }
  </style>
</head>
<body>

<header class="top">
  <div class="brand">BreakBite</div>
  <nav class="tabs" aria-label="Main sections">
    <a class="tab" href="student.html">Order</a>
    <a class="tab" href="planner.html">Break Planner</a>
    <a class="tab active" href="index.html">Home</a>
  </nav>
</header>

<main class="wide">
  <div class="role-switch">
    <a class="role-btn on" href="index.html">
      <span class="ico">S</span> Student
    </a>
    <a class="role-btn" href="staff.html">
      <span class="ico">T</span> Staff
    </a>
  </div>

  <section class="hero">
    <h1>Less waiting, more break.</h1>
    <p>Pre-order your canteen food, skip the queue, and plan your break with the time you have.</p>
  </section>

  <div class="grid">
    <a class="card" href="student.html">
      <h2>Order food</h2>
      <p>See today's menu, pick your items, and track your token until it is ready.</p>
      <span class="go">Open ordering &rarr;</span>
    </a>
    <a class="card" href="planner.html">
      <h2>Break Planner</h2>
      <p>Enter your break time and money. We suggest what you can eat in that time.</p>
      <span class="go">Plan my break &rarr;</span>
    </a>
  </div>
</main>

</body>
</html>
```

B.2 firebase.js (placeholders)

Replace the placeholder values with the six values from your Firebase web app settings.

```
// firebase.js: the only place where the Firebase settings live.
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-app.js";
import { getFirestore } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";
import { getAuth } from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.firebasestorage.app",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID"
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);
```

B.3 student.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite - Order</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header class="top">
  <div class="brand">BreakBite</div>
  <nav class="tabs" aria-label="Student sections">
    <a class="tab active" href="student.html">Order</a>
    <a class="tab" href="planner.html">Break Planner</a>
    <a class="tab" href="index.html">Home</a>
  </nav>
</header>

<main class="narrow">
  <section class="hero">
    <h1>Pre-order, skip the queue.</h1>
    <p class="muted">Smart Food, Smart Time. Pick your food, place the order, and collect it when your
        token is ready.</p>
  </section>

  <div class="card rush">
    <div>
      <b>Canteen rush</b>
      <div class="muted" id="rush-text">Checking...</div>
    </div>
    <div class="num" id="rush-num">-</div>
  </div>

  <div class="card">
    <h2>Today's menu</h2>
    <div id="menu-list"><div class="empty">Loading menu...</div></div>
  </div>

  <div class="cart">
    <span id="cart-text">No items selected</span>
    <button id="order-btn" disabled>Place pre-order</button>
  </div>

  <div class="card" id="orders-card" hidden>
    <h2>Your orders</h2>
    <div id="my-orders"></div>
  </div>

  <div class="card">
    <h2>Check a token</h2>
    <p class="muted">Forgot your status? Enter your token number.</p>
    <div class="form-row">
      <input type="number" id="lookup-token" placeholder="Token number" min="1">
      <button id="lookup-btn">Check status</button>
    </div>
    <div id="lookup-result"></div>
  </div>
</main>

<div id="toast" hidden></div>

<script type="module">
import { db } from "./firebase.js";
import {
  collection, doc, onSnapshot, query, orderBy, where, getDocs, runTransaction, serverTimestamp
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";

const STEPS = ["Received", "Preparing", "Ready"];
const KEY = "breakbite_orders";      // list of order IDs this browser placed
const OLD_KEY = "breakbite_order";   // single ID used by the earlier version

let menu = [];
let qty = {};
let pending = 0;
let live = {};       // orderId -> latest order data
let watching = {};   // orderId -> unsubscribe function
```

```
const $ = function (id) { return document.getElementById(id); };

function esc(s) {
  return String(s).replace(/[&<>"']/g, function (c) {
    return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
  });
}

function price(f) {
  return Math.round(f.price * (100 - (f.discount || 0)) / 100);
}

function toast(msg) {
  const t = $("toast");
  t.textContent = msg;
  t.hidden = false;
  clearTimeout(toast.timer);
  toast.timer = setTimeout(function () { t.hidden = true; }, 2500);
}

function waitMinutes() {
  return 3 + pending * 2;
}

function cartItems() {
  return menu
    .filter(function (f) { return qty[f.id] > 0; })
    .map(function (f) { return { name: f.name, qty: qty[f.id], price: price(f) }; });
}

// ---------- saved order IDs ----------
function loadIds() {
  try { return JSON.parse(localStorage.getItem(KEY)) || []; } catch (e) { return []; }
}
function saveIds(ids) {
  localStorage.setItem(KEY, JSON.stringify(ids));
}
function addId(id) {
  const ids = loadIds();
  if (ids.indexOf(id) < 0) ids.push(id);
  saveIds(ids);
}
function removeId(id) {
  saveIds(loadIds().filter(function (x) { return x !== id; }));
  if (watching[id]) { watching[id](); delete watching[id]; }
  delete live[id];
  renderMyOrders();
}

// Move an order saved by the earlier version into the new list
(function migrate() {
  const old = localStorage.getItem(OLD_KEY);
  if (old) {
    addId(old);
    localStorage.removeItem(OLD_KEY);
  }
})();

// ---------- status helpers ----------
function stepsHtml(status) {
  const idx = status === "Done" ? STEPS.length : STEPS.indexOf(status);
  return '<div class="steps">' + STEPS.map(function (s, i) {
    const cls = i < idx ? " done" : (i === idx ? " now" : "");
    return '<div class="step' + cls + '"><span>' + (i + 1) + '</span>' + s + '</div>';
  }).join("") + '</div>';
}

function statusMessage(status) {
  if (status === "Ready") return "Ready! Please collect your food.";
  if (status === "Preparing") return "Your food is being prepared.";
  if (status === "Done") return "Thank you! Order collected.";
  return "Waiting. Estimated " + waitMinutes() + " min.";
}

function itemsText(o) {
  return o.items.map(function (i) { return i.name + " x" + i.qty; }).join(", ");
}
```

```
// ---------- Menu ----------
onSnapshot(query(collection(db, "menu"), orderBy("order")),
  function (snap) {
    menu = snap.docs
      .map(function (d) { return Object.assign({ id: d.id }, d.data()); })
      .filter(function (f) { return f.available !== false; });
    drawMenu();
    updateCart();
  },
  function (err) {
    $("menu-list").innerHTML = '<div class="empty">Could not load the menu. Check the Firebase
        setup.</div>';
    console.error(err);
  }
);

function drawMenu() {
  if (!menu.length) {
    $("menu-list").innerHTML = '<div class="empty">No items right now. Please check again later.</div>';
    return;
  }
  $("menu-list").innerHTML = menu.map(function (f) {
    const p = price(f);
    const priceHtml = f.discount > 0
      ? '<s class="muted">' + f.price + ' Tk</s> <b>' + p + ' Tk</b> <span class="badge">' + f.discount
          + '% off</span>'
      : '<b>' + f.price + ' Tk</b>';
    const q = qty[f.id] || 0;
    return '<div class="item">' +
      '<div><div class="name">' + esc(f.name) + '</div><div>' + priceHtml + '</div></div>' +
      '<div class="stepper">' +
        '<button class="ghost small" data-act="dec" data-id="' + f.id + '">-</button>' +
        '<span>' + q + '</span>' +
        '<button class="small" data-act="inc" data-id="' + f.id + '">+</button>' +
      '</div></div>';
  }).join("");
}

$("menu-list").addEventListener("click", function (e) {
  const b = e.target.closest("button[data-act]");
  if (!b) return;
  const id = b.dataset.id;
  const cur = qty[id] || 0;
  qty[id] = b.dataset.act === "inc" ? Math.min(cur + 1, 20) : Math.max(cur - 1, 0);
  if (qty[id] === 0) delete qty[id];
  drawMenu();
  updateCart();
});

function updateCart() {
  const items = cartItems();
  const count = items.reduce(function (a, i) { return a + i.qty; }, 0);
  const total = items.reduce(function (a, i) { return a + i.qty * i.price; }, 0);
  $("cart-text").textContent = count ? count + " item(s), " + total + " Tk" : "No items selected";
  $("order-btn").disabled = count === 0;
}

// ---------- Rush radar ----------
onSnapshot(query(collection(db, "orders"), where("status", "in", ["Received", "Preparing"])),
  function (snap) {
    pending = snap.size;
    let text;
    if (pending >= 10) text = "High rush. Pre-order to save time.";
    else if (pending >= 5) text = "Medium rush. Pre-order if you can.";
    else text = "Low rush. Good time to visit.";
    $("rush-text").textContent = text + " " + pending + " order(s) waiting.";
    $("rush-num").textContent = pending;
  },
  function (err) { console.error(err); }
);

// ---------- Your orders (all active orders from this browser) ----------
function renderMyOrders() {
  const ids = Object.keys(live);
  $("orders-card").hidden = ids.length === 0;
  $("my-orders").innerHTML = ids.map(function (id) {
    const o = live[id];
    return '<div class="card">' +
```

```
      '<div class="ticket-big">' +
        '<small>Your token</small>' +
        '<div class="tok">' + o.token + '</div>' +
        '<div class="muted">' + esc(itemsText(o)) + '</div>' +
      '</div>' +
      stepsHtml(o.status) +
      '<p class="track-msg">' + statusMessage(o.status) + '</p>' +
    '</div>';
  }).join("");
}

function watch(id) {
  if (watching[id]) return;
  watching[id] = onSnapshot(doc(db, "orders", id),
    function (snap) {
      if (!snap.exists()) { removeId(id); return; }
      const o = snap.data();
      if (o.status === "Done") {
        toast("Token " + o.token + " collected. Thank you!");
        removeId(id);
      } else {
        live[id] = o;
        renderMyOrders();
      }
    },
    function (err) { console.error(err); }
  );
}

// ---------- Place order ----------
$("order-btn").addEventListener("click", async function () {
  const items = cartItems();
  if (!items.length) return;
  const count = items.reduce(function (a, i) { return a + i.qty; }, 0);
  const total = items.reduce(function (a, i) { return a + i.qty * i.price; }, 0);

  const btn = $("order-btn");
  btn.disabled = true;
  btn.textContent = "Placing...";
  try {
    const counterRef = doc(db, "counters", "orders");
    const orderRef = doc(collection(db, "orders"));
    await runTransaction(db, async function (tx) {
      const cs = await tx.get(counterRef);
      const next = (cs.exists() ? cs.data().last : 0) + 1;
      tx.set(counterRef, { last: next });
      tx.set(orderRef, {
        token: next,
        items: items,
        count: count,
        total: total,
        status: "Received",
        createdAt: serverTimestamp()
      });
    });
    addId(orderRef.id);
    qty = {};
    drawMenu();
    updateCart();
    watch(orderRef.id);
    toast("Order placed!");
  } catch (err) {
    toast("Could not place the order. Check your internet and try again.");
    console.error(err);
  }
  btn.textContent = "Place pre-order";
  updateCart();
});

// ---------- Check any token ----------
$("lookup-btn").addEventListener("click", async function () {
  const n = parseInt($("lookup-token").value, 10);
  if (isNaN(n) || n < 1) { toast("Enter a valid token number."); return; }
  try {
    const snap = await getDocs(query(collection(db, "orders"), where("token", "==", n)));
    if (snap.empty) {
      $("lookup-result").innerHTML = '<p class="muted">Token ' + n + ' not found.</p>';
      return;
```

```
    }
    const o = snap.docs[0].data();
    $("lookup-result").innerHTML =
      '<div class="ticket-big"><small>Token</small><div class="tok">' + o.token + '</div>' +
      '<div class="muted">' + esc(itemsText(o)) + '</div></div>' +
      stepsHtml(o.status) +
      '<p class="track-msg">' + statusMessage(o.status) + '</p>';
  } catch (err) {
    toast("Could not check the token. Try again.");
    console.error(err);
  }
});

// ---------- Start: reconnect every order saved in this browser ----------
loadIds().forEach(watch);
</script>
</body>
</html>
```

B.4 planner.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite - Break Planner</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header class="top">
  <div class="brand">BreakBite</div>
  <nav class="tabs" aria-label="Main sections">
    <a class="tab" href="student.html">Order</a>
    <a class="tab active" href="planner.html">Break Planner</a>
    <a class="tab" href="index.html">Home</a>
  </nav>
</header>

<main class="narrow">
  <h1>Break Planner</h1>
  <p class="intro">Tell us how long your break is and how much money you have. We will suggest what you
      can eat.</p>

  <div class="card">
    <h2>Your break</h2>
    <div class="form-grid">
      <label>Break time (minutes)
        <input type="number" id="time" placeholder="e.g. 15">
      </label>
      <label>Money (Tk)
        <input type="number" id="money" min="0" placeholder="e.g. 100">
      </label>
    </div>
    <button id="plan-btn" disabled>Loading menu...</button>
    <p class="error" id="error" role="alert"></p>
  </div>

  <div class="card" id="result-card" hidden>
    <h2>Suggested for you</h2>
    <div class="summary">
      <span id="result-info"></span>
      <b id="result-total"></b>
    </div>
    <div id="result-list"></div>
  </div>
</main>

<script type="module">
import { db } from "./firebase.js";
import {
  collection, doc, getDoc, getDocs, query, orderBy
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";

const DEFAULTS = { minBreak: 10, maxBreak: 45, step: 5, defaultEat: 5 };

let settings = Object.assign({}, DEFAULTS);
let foods = [];

const $ = function (id) { return document.getElementById(id); };

function esc(s) {
  return String(s).replace(/[&<>"']/g, function (c) {
    return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
  });
}

function showError(msg) {
  $("error").textContent = msg;
  if (msg) $("result-card").hidden = true;
}

function finalPrice(f) {
  return Math.round(f.price * (100 - (f.discount || 0)) / 100);
}

// ---------- load settings and menu once ----------
async function load() {
```

```
  try {
    const sSnap = await getDoc(doc(db, "settings", "planner"));
    if (sSnap.exists()) settings = Object.assign({}, DEFAULTS, sSnap.data());
  } catch (err) {
    console.error(err); // fall back to defaults
  }

  try {
    const snap = await getDocs(query(collection(db, "menu"), orderBy("order")));
    foods = snap.docs
      .map(function (d) { return Object.assign({ id: d.id }, d.data()); })
      .filter(function (f) { return f.available !== false; })
      .map(function (f) {
        return {
          name: f.name,
          price: finalPrice(f),
          oldPrice: f.price,
          discount: f.discount || 0,
          minutes: f.minutes > 0 ? f.minutes : settings.defaultEat
        };
      })
      .filter(function (f) { return f.price >= 0 && f.minutes > 0; });
  } catch (err) {
    console.error(err);
    showError("Could not load the menu. Check your internet and refresh.");
    $("plan-btn").textContent = "Show options";
    return;
  }

  const t = $("time");
  t.min = settings.minBreak;
  t.max = settings.maxBreak;
  t.step = settings.step;
  t.placeholder = "e.g. " + Math.min(settings.maxBreak, Math.max(settings.minBreak, 15));

  $("plan-btn").disabled = false;
  $("plan-btn").textContent = "Show options";
  if (!foods.length) showError("No food is available right now. Please check again later.");
}

// ---------- best combination ----------
// Most items that fit the time and money. Ties go to the bigger spend.
// Same item can be picked more than once.
function bestCombo(minutes, money) {
  const cap = Math.min(money, 1500);
  const W = cap + 1;
  const size = (minutes + 1) * W;
  const count = new Int16Array(size).fill(-1);
  const from = new Int16Array(size).fill(-1);
  count[0] = 0;

  for (let i = 0; i < foods.length; i++) {
    const ft = foods[i].minutes, fp = foods[i].price;
    if (ft > minutes || fp > cap) continue;
    for (let t = ft; t <= minutes; t++) {
      for (let m = fp; m <= cap; m++) {
        const prev = (t - ft) * W + (m - fp);
        if (count[prev] < 0) continue;
        const idx = t * W + m;
        if (count[prev] + 1 > count[idx]) {
          count[idx] = count[prev] + 1;
          from[idx] = i;
        }
      }
    }
  }

  let bestIdx = 0, bestCount = 0, bestSpend = 0;
  for (let t = 0; t <= minutes; t++) {
    for (let m = 0; m <= cap; m++) {
      const c = count[t * W + m];
      if (c > bestCount || (c === bestCount && c > 0 && m > bestSpend)) {
        bestCount = c; bestSpend = m; bestIdx = t * W + m;
      }
    }
  }

  const picked = [];
```

```
  let idx = bestIdx;
  let guard = 0;
  while (idx > 0 && from[idx] >= 0 && guard++ < 200) {
    const f = foods[from[idx]];
    picked.push(f);
    const t = Math.floor(idx / W), m = idx % W;
    idx = (t - f.minutes) * W + (m - f.price);
  }
  return picked;
}

function groupItems(items) {
  const map = {};
  items.forEach(function (f) {
    if (!map[f.name]) map[f.name] = { food: f, qty: 0 };
    map[f.name].qty++;
  });
  return Object.keys(map).map(function (k) { return map[k]; });
}

$("plan-btn").addEventListener("click", function () {
  const minutes = parseInt($("time").value, 10);
  const money = parseInt($("money").value, 10);

  if (isNaN(minutes) || minutes < settings.minBreak || minutes > settings.maxBreak) {
    showError("Please enter a break time between " + settings.minBreak + " and " + settings.maxBreak + "
        minutes.");
    return;
  }
  if (isNaN(money) || money < 0) {
    showError("Please enter how much money you have.");
    return;
  }
  if (!foods.length) {
    showError("No food is available right now. Please check again later.");
    return;
  }
  showError("");

  const items = bestCombo(minutes, money);
  const list = $("result-list");

  if (!items.length) {
    list.innerHTML = '<div class="empty">Nothing fits in ' + minutes + ' minutes and ' + money +
      ' Tk. Try more time or money.</div>';
    $("result-info").textContent = minutes + " min, " + money + " Tk";
    $("result-total").textContent = "";
  } else {
    const totalTime = items.reduce(function (a, f) { return a + f.minutes; }, 0);
    const totalCost = items.reduce(function (a, f) { return a + f.price; }, 0);
    list.innerHTML = groupItems(items).map(function (g) {
      const f = g.food;
      const off = f.discount > 0 ? ' <span class="badge">' + f.discount + '% off</span>' : '';
      return '<div class="item"><div><div class="name">' + esc(f.name) + ' x' + g.qty + off +
        '</div><div class="meta">' + f.minutes + ' min each</div></div>' +
        '<div class="price">' + (f.price * g.qty) + ' Tk</div></div>';
    }).join("");
    $("result-info").textContent = "Time used: " + totalTime + " of " + minutes + " min";
    $("result-total").textContent = "Total: " + totalCost + " Tk";
  }
  $("result-card").hidden = false;
});

load();
</script>

</body>
</html>
```

B.5 staff.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BreakBite Staff</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<!-- ================= LOGIN ================= -->
<div id="login-screen">
  <div class="login-brand">
    <div class="logo">BreakBite</div>
    <p>Smart Food, Smart Time. Manage your canteen orders and menu from one place.</p>
  </div>
  <div class="login-form">
    <form id="login-form">
      <h2>Staff login</h2>
      <p class="muted">Only canteen staff can open this page.</p>
      <label class="field">Email
        <input type="email" id="username" autocomplete="username" required>
      </label>
      <label class="field">Password
        <input type="password" id="password" autocomplete="current-password" required>
      </label>
      <div class="msg" id="login-error" role="alert"></div>
      <button type="submit">Log in</button>
    </form>
  </div>
</div>

<!-- ================= APP ================= -->
<div id="app" class="hidden">
  <header class="top">
    <div class="brand">BreakBite</div>
    <nav class="tabs" aria-label="Staff sections">
      <button class="tab active" data-tab="orders">Orders</button>
      <button class="tab" data-tab="menu">Menu</button>
      <button class="tab" data-tab="waste">Food Waste Saver</button>
      <button class="tab" data-tab="settings">Settings</button>
    </nav>
    <div class="right"><button class="ghost small" id="logout-btn">Log out</button></div>
  </header>

  <main>
    <!-- Orders -->
    <section class="panel active" id="panel-orders">
      <h2>Orders</h2>
      <div class="stats">
        <div class="stat" id="wait-stat">
          <div class="muted">Orders in queue</div>
          <div class="num" id="queue-count">0</div>
        </div>
        <div class="stat">
          <div class="muted">Estimated waiting time</div>
          <div class="num" id="wait-time">3 min</div>
        </div>
        <div class="stat">
          <div class="muted">Ready to collect</div>
          <div class="num" id="ready-count">0</div>
        </div>
      </div>

      <div class="board">
        <div class="col">
          <h3>Received <span class="count" id="cnt-Received">0</span></h3>
          <div id="col-Received"></div>
        </div>
        <div class="col">
          <h3>Preparing <span class="count" id="cnt-Preparing">0</span></h3>
          <div id="col-Preparing"></div>
        </div>
        <div class="col">
          <h3>Ready <span class="count" id="cnt-Ready">0</span></h3>
          <div id="col-Ready"></div>
        </div>
      </div>
```

```
      <div class="card">
        <h3>Completed (latest 10)</h3>
        <div id="done-list"></div>
      </div>

      <div class="card">
        <h3>End of day</h3>
        <p class="muted">Saves today's report, removes all orders, and restarts token numbers from 1.
            Press this once at the end of the day.</p>
        <button class="danger" id="eod-btn">End of day: save report and reset tokens</button>
      </div>

      <div class="card">
        <h3>Past daily reports</h3>
        <div id="reports-list"></div>
      </div>
    </section>

    <!-- Menu -->
    <section class="panel" id="panel-menu">
      <h2>Menu</h2>
      <div class="card">
        <div class="table-wrap">
          <table>
            <thead>
              <tr><th>Item</th><th>Price</th><th>Eat time</th><th>Status</th><th>Actions</th></tr>
            </thead>
            <tbody id="menu-body"></tbody>
          </table>
        </div>
      </div>
      <div class="card">
        <h3>Add a new item</h3>
        <div class="form-row">
          <input type="text" id="food-name" placeholder="Item name">
          <input type="number" id="food-price" placeholder="Price (Tk)" min="0">
          <input type="number" id="food-minutes" placeholder="Eat time (min)" min="1">
          <button id="add-food-btn">Add item</button>
        </div>
      </div>
    </section>

    <!-- Food Waste Saver -->
    <section class="panel" id="panel-waste">
      <h2>Food Waste Saver</h2>
      <div class="card">
        <h3>Start a discount</h3>
        <div class="form-row">
          <select id="waste-food"></select>
          <input type="number" id="waste-percent" placeholder="Discount %" min="5" max="90">
          <button id="waste-start">Start discount</button>
        </div>
      </div>
      <div class="card">
        <h3>Running discounts</h3>
        <div id="waste-list"></div>
      </div>
    </section>

    <!-- Settings -->
    <section class="panel" id="panel-settings">
      <h2>Settings</h2>

      <div class="card">
        <h3>Break Planner</h3>
        <p class="muted">These values control the Break Planner page for students.</p>
        <div class="grid2">
          <label class="field">Shortest break (min)
            <input type="number" id="pl-min" min="1">
          </label>
          <label class="field">Longest break (min)
            <input type="number" id="pl-max" min="1">
          </label>
          <label class="field">Time step (min)
            <input type="number" id="pl-step" min="1">
          </label>
          <label class="field">Default eat time if missing (min)
```

```
            <input type="number" id="pl-default" min="1">
          </label>
        </div>
        <button id="planner-save">Save planner settings</button>
      </div>

      <div class="card">
        <h3>Change password</h3>
        <p class="muted">Password is managed through Firebase Authentication. Use the Firebase console
            to reset it.</p>
      </div>
    </section>
  </main>
</div>

<div id="toast" class="hidden"></div>

<script type="module">
import { db, auth } from "./firebase.js";
import {
  collection, doc, addDoc, updateDoc, deleteDoc, setDoc, getDoc, getDocs,
  onSnapshot, query, orderBy, limit, writeBatch, serverTimestamp
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore.js";
import {
  signInWithEmailAndPassword, signOut, onAuthStateChanged
} from "https://www.gstatic.com/firebasejs/10.12.2/firebase-auth.js";

const STATUS_FLOW = ["Received", "Preparing", "Ready", "Done"];
const NEXT_LABEL = { Received: "Start preparing", Preparing: "Mark ready", Ready: "Mark collected" };
const DEFAULT_EAT_MIN = 5;

let menu = [];
let orders = [];
let started = false;

const $ = function (id) { return document.getElementById(id); };

function escapeHtml(s) {
  return String(s).replace(/[&<>"']/g, function (c) {
    return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
  });
}

function toast(msg) {
  const t = $("toast");
  t.textContent = msg;
  t.classList.remove("hidden");
  clearTimeout(toast.timer);
  toast.timer = setTimeout(function () { t.classList.add("hidden"); }, 2500);
}

function finalPrice(f) {
  return Math.round(f.price * (100 - (f.discount || 0)) / 100);
}

function itemsText(o) {
  return o.items.map(function (i) { return i.name + " x" + i.qty; }).join(", ");
}

function timeText(o) {
  const d = o.createdAt && o.createdAt.toDate ? o.createdAt.toDate() : new Date();
  return d.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" });
}

function todayKey() {
  return new Date().toLocaleDateString("en-CA"); // YYYY-MM-DD in local time
}

// ---------- login ----------
onAuthStateChanged(auth, function (user) {
  if (user) {
    $("login-screen").classList.add("hidden");
    $("app").classList.remove("hidden");
    if (!started) {
      started = true;
      watchOrders();
      watchMenu();
      watchReports();
```

```
      loadPlannerSettings();
    }
  } else {
    $("login-screen").classList.remove("hidden");
    $("app").classList.add("hidden");
  }
});

$("login-form").addEventListener("submit", async function (e) {
  e.preventDefault();
  $("login-error").textContent = "";
  try {
    await signInWithEmailAndPassword(auth, $("username").value.trim(), $("password").value);
    $("password").value = "";
  } catch (err) {
    $("login-error").textContent = "Wrong email or password. Try again.";
    console.error(err);
  }
});

$("logout-btn").addEventListener("click", function () { signOut(auth); });

// ---------- tabs ----------
function showTab(name) {
  document.querySelectorAll(".tab").forEach(function (b) {
    b.classList.toggle("active", b.dataset.tab === name);
  });
  document.querySelectorAll(".panel").forEach(function (p) {
    p.classList.toggle("active", p.id === "panel-" + name);
  });
}
document.querySelectorAll(".tab").forEach(function (b) {
  b.addEventListener("click", function () { showTab(b.dataset.tab); });
});

// ---------- one click handler for dynamic buttons ----------
document.addEventListener("click", function (e) {
  const el = e.target.closest("[data-act]");
  if (!el) return;
  const id = el.dataset.id;
  const act = el.dataset.act;
  if (act === "advance") advanceOrder(id);
  if (act === "toggle") toggleAvailable(id);
  if (act === "edit") editFood(id);
  if (act === "delete") deleteFood(id);
  if (act === "stopDiscount") setDiscount(id, 0);
});

// ---------- orders ----------
function watchOrders() {
  onSnapshot(query(collection(db, "orders"), orderBy("createdAt", "desc"), limit(50)),
    function (snap) {
      orders = snap.docs.map(function (d) { return Object.assign({ id: d.id }, d.data()); });
      renderOrders();
    },
    function (err) { toast("Could not load orders."); console.error(err); }
  );
}

function renderOrders() {
  const active = orders.filter(function (o) { return o.status !== "Done"; });
  const queue = active.filter(function (o) { return o.status === "Received" || o.status === "Preparing";
      }).length;
  const ready = active.filter(function (o) { return o.status === "Ready"; }).length;

  $("queue-count").textContent = queue;
  $("wait-time").textContent = (3 + queue * 2) + " min";
  $("ready-count").textContent = ready;
  $("wait-stat").classList.toggle("hot", queue >= 8);

  ["Received", "Preparing", "Ready"].forEach(function (s) {
    const list = active.filter(function (o) { return o.status === s; });
    $("cnt-" + s).textContent = list.length;
    $("col-" + s).innerHTML = list.length
      ? list.map(ticketHtml).join("")
      : '<div class="empty">No ' + s.toLowerCase() + ' orders.</div>';
  });
```

```
  const done = orders.filter(function (o) { return o.status === "Done"; }).slice(0, 10);
  $("done-list").innerHTML = done.length
    ? done.map(function (o) {
        return '<div class="list-row">' +
          '<span><span class="badge off">#' + o.token + '</span> ' + escapeHtml(itemsText(o)) +
              '</span>' +
          '<span class="muted">' + o.total + ' Tk, ' + timeText(o) + '</span>' +
        '</div>';
      }).join("")
    : '<div class="empty">Nothing completed yet.</div>';
}

function ticketHtml(o) {
  return '<div class="ticket ' + o.status + '">' +
    '<div class="stub"><small>Token</small><b>' + o.token + '</b></div>' +
    '<div class="body">' +
      '<div class="items">' + escapeHtml(itemsText(o)) + '</div>' +
      '<div class="meta">' + o.total + ' Tk, ordered at ' + timeText(o) + '</div>' +
      '<button class="small" data-act="advance" data-id="' + o.id + '">' + NEXT_LABEL[o.status] +
          '</button>' +
    '</div></div>';
}

function advanceOrder(id) {
  const o = orders.find(function (x) { return x.id === id; });
  if (!o) return;
  const next = STATUS_FLOW[STATUS_FLOW.indexOf(o.status) + 1];
  if (!next) return;
  updateDoc(doc(db, "orders", id), { status: next }).catch(function (err) {
    toast("Could not update the order.");
    console.error(err);
  });
}

// ---------- end of day ----------
$("eod-btn").addEventListener("click", endOfDay);

async function endOfDay() {
  let all;
  try {
    all = await getDocs(collection(db, "orders"));
  } catch (err) {
    toast("Could not read orders. Try again.");
    console.error(err);
    return;
  }

  if (all.empty) {
    toast("No orders to close today.");
    return;
  }

  const activeCount = all.docs.filter(function (d) { return d.data().status !== "Done"; }).length;
  let msg = "Save today's report and remove all " + all.size + " orders? Token numbers will restart from
      1.";
  if (activeCount > 0) {
    msg += "\n\n" + activeCount + " order(s) are still active and will also be removed.";
  }
  if (!confirm(msg)) return;
  if (!confirm("This cannot be undone. Are you sure?")) return;

  // Build the summary before anything is deleted
  let totalSales = 0;
  const itemCount = {};
  all.docs.forEach(function (d) {
    const o = d.data();
    totalSales += Number(o.total) || 0;
    (o.items || []).forEach(function (i) {
      itemCount[i.name] = (itemCount[i.name] || 0) + (Number(i.qty) || 0);
    });
  });

  const btn = $("eod-btn");
  btn.disabled = true;
  btn.textContent = "Closing the day...";

  try {
    // 1. Save the report first. If this fails, nothing is deleted.
```

```
    await addDoc(collection(db, "daily_reports"), {
      date: todayKey(),
      totalOrders: all.size,
      totalSales: totalSales,
      items: itemCount,
      activeAtClose: activeCount,
      createdAt: serverTimestamp()
    });

    // 2. Delete all orders in batches (Firestore allows up to 500 per batch)
    const docs = all.docs;
    for (let i = 0; i < docs.length; i += 400) {
      const batch = writeBatch(db);
      docs.slice(i, i + 400).forEach(function (d) { batch.delete(d.ref); });
      await batch.commit();
    }

    // 3. Restart token numbers from 1
    await setDoc(doc(db, "counters", "orders"), { last: 0 });

    toast("Day closed. Report saved and tokens restart from 1.");
  } catch (err) {
    toast("Could not close the day. Nothing was deleted if the report failed. Try again.");
    console.error(err);
  }

  btn.disabled = false;
  btn.textContent = "End of day: save report and reset tokens";
}

// ---------- past reports ----------
function watchReports() {
  onSnapshot(query(collection(db, "daily_reports"), orderBy("createdAt", "desc"), limit(7)),
    function (snap) {
      if (snap.empty) {
        $("reports-list").innerHTML = '<div class="empty">No reports yet. A report is saved each time
            you press End of day.</div>';
        return;
      }
      $("reports-list").innerHTML = snap.docs.map(function (d) {
        const r = d.data();
        const top = Object.keys(r.items || {})
          .map(function (name) { return { name: name, qty: r.items[name] }; })
          .sort(function (a, b) { return b.qty - a.qty; })
          .slice(0, 5)
          .map(function (i) { return i.name + " x" + i.qty; })
          .join(", ");
        return '<div class="list-row">' +
          '<span><b>' + escapeHtml(r.date || "") + '</b>: ' + (r.totalOrders || 0) + ' orders, ' +
          (r.totalSales || 0) + ' Tk<br><span class="muted">Top items: ' + escapeHtml(top || "none") +
              '</span></span>' +
        '</div>';
      }).join("");
    },
    function (err) { console.error(err); }
  );
}

// ---------- menu ----------
function watchMenu() {
  onSnapshot(query(collection(db, "menu"), orderBy("order")),
    function (snap) {
      menu = snap.docs.map(function (d) { return Object.assign({ id: d.id }, d.data()); });
      renderMenu();
      renderWaste();
    },
    function (err) { toast("Could not load the menu."); console.error(err); }
  );
}

function renderMenu() {
  const rows = menu.map(function (f) {
    const price = f.discount > 0
      ? '<s class="off-text">' + f.price + '</s> <b>' + finalPrice(f) + ' Tk</b> <span class="badge">' +
          f.discount + '% off</span>'
      : f.price + ' Tk';
    const eat = (f.minutes || DEFAULT_EAT_MIN) + ' min';
    return '<tr>' +
```

```
      '<td data-label="Item">' + escapeHtml(f.name) + '</td>' +
      '<td data-label="Price">' + price + '</td>' +
      '<td data-label="Eat time">' + eat + '</td>' +
      '<td data-label="Status"><span class="badge ' + (f.available ? "green" : "off") + '">' +
          (f.available ? "Available" : "Sold out") + '</span></td>' +
      '<td class="actions">' +
        '<button class="ghost small" data-act="toggle" data-id="' + f.id + '">' + (f.available ? "Mark
            sold out" : "Mark available") + '</button>' +
        '<button class="ghost small" data-act="edit" data-id="' + f.id + '">Edit</button>' +
        '<button class="danger small" data-act="delete" data-id="' + f.id + '">Delete</button>' +
      '</td></tr>';
  });
  $("menu-body").innerHTML = rows.length
    ? rows.join("")
    : '<tr><td colspan="5" class="empty">No items yet. Add one below.</td></tr>';
}

async function addFood() {
  const name = $("food-name").value.trim();
  const price = parseInt($("food-price").value, 10);
  const minutes = parseInt($("food-minutes").value, 10);
  if (!name || isNaN(price) || price < 0) {
    toast("Enter a food name and a valid price.");
    return;
  }
  try {
    await addDoc(collection(db, "menu"), {
      name: name,
      price: price,
      minutes: isNaN(minutes) || minutes < 1 ? DEFAULT_EAT_MIN : minutes,
      available: true,
      discount: 0,
      order: Date.now()
    });
    $("food-name").value = "";
    $("food-price").value = "";
    $("food-minutes").value = "";
    toast(name + " added to the menu.");
  } catch (err) {
    toast("Could not add the item.");
    console.error(err);
  }
}

function toggleAvailable(id) {
  const f = menu.find(function (x) { return x.id === id; });
  if (!f) return;
  updateDoc(doc(db, "menu", id), { available: !f.available });
}

function editFood(id) {
  const f = menu.find(function (x) { return x.id === id; });
  if (!f) return;
  const name = prompt("Food name:", f.name);
  if (name === null) return;
  const priceText = prompt("Price (Tk):", f.price);
  if (priceText === null) return;
  const minText = prompt("Eat time (minutes):", f.minutes || DEFAULT_EAT_MIN);
  if (minText === null) return;
  const price = parseInt(priceText, 10);
  const minutes = parseInt(minText, 10);
  if (!name.trim() || isNaN(price) || price < 0 || isNaN(minutes) || minutes < 1) {
    toast("Name, price or eat time is not valid.");
    return;
  }
  updateDoc(doc(db, "menu", id), { name: name.trim(), price: price, minutes: minutes });
  toast("Menu item updated.");
}

function deleteFood(id) {
  const f = menu.find(function (x) { return x.id === id; });
  if (!f) return;
  if (!confirm("Remove " + f.name + " from the menu?")) return;
  deleteDoc(doc(db, "menu", id));
}

$("add-food-btn").addEventListener("click", addFood);
```

```
// ---------- food waste saver ----------
function renderWaste() {
  const available = menu.filter(function (f) { return f.available; });
  $("waste-food").innerHTML = available.map(function (f) {
    return '<option value="' + f.id + '">' + escapeHtml(f.name) + ' (' + f.price + ' Tk)</option>';
  }).join("");

  const discounted = menu.filter(function (f) { return f.discount > 0; });
  $("waste-list").innerHTML = discounted.length
    ? discounted.map(function (f) {
        return '<div class="list-row">' +
          '<span><b>' + escapeHtml(f.name) + '</b>: ' + f.discount + '% off, now ' + finalPrice(f) + '
              Tk</span>' +
          '<button class="ghost small" data-act="stopDiscount" data-id="' + f.id + '">Stop
              discount</button>' +
        '</div>';
      }).join("")
    : '<div class="empty">No discounts running. Pick a food above to start one.</div>';
}

function setDiscount(id, percent) {
  updateDoc(doc(db, "menu", id), { discount: percent }).catch(function (err) {
    toast("Could not save the discount.");
    console.error(err);
  });
}

$("waste-start").addEventListener("click", function () {
  const id = $("waste-food").value;
  const percent = parseInt($("waste-percent").value, 10);
  if (!id) { toast("Choose a food item first."); return; }
  if (isNaN(percent) || percent < 5 || percent > 90) {
    toast("Discount must be between 5 and 90.");
    return;
  }
  setDiscount(id, percent);
  toast("Discount started.");
});

// ---------- planner settings ----------
const PLANNER_DEFAULTS = { minBreak: 10, maxBreak: 45, step: 5, defaultEat: 5 };

async function loadPlannerSettings() {
  try {
    const snap = await getDoc(doc(db, "settings", "planner"));
    const s = Object.assign({}, PLANNER_DEFAULTS, snap.exists() ? snap.data() : {});
    $("pl-min").value = s.minBreak;
    $("pl-max").value = s.maxBreak;
    $("pl-step").value = s.step;
    $("pl-default").value = s.defaultEat;
  } catch (err) {
    console.error(err);
  }
}

$("planner-save").addEventListener("click", async function () {
  const minBreak = parseInt($("pl-min").value, 10);
  const maxBreak = parseInt($("pl-max").value, 10);
  const step = parseInt($("pl-step").value, 10);
  const defaultEat = parseInt($("pl-default").value, 10);
  if (isNaN(minBreak) || isNaN(maxBreak) || minBreak < 1 || maxBreak <= minBreak) {
    toast("Longest break must be bigger than shortest break.");
    return;
  }
  if (isNaN(step) || step < 1 || isNaN(defaultEat) || defaultEat < 1) {
    toast("Step and default eat time must be at least 1.");
    return;
  }
  try {
    await setDoc(doc(db, "settings", "planner"), {
      minBreak: minBreak, maxBreak: maxBreak, step: step, defaultEat: defaultEat
    });
    toast("Planner settings saved.");
  } catch (err) {
    toast("Could not save planner settings.");
    console.error(err);
  }
});
```

```
</script>
</body>
</html>
```

B.6 style.css

```
/* ========================================================
   BreakBite shared stylesheet: "Spice Kitchen" theme
   Change only the colours in :root to re-theme the whole site.
   Food doodles and animations are included; no HTML change needed.
   ======================================================== */

:root {
  /* ---- Main colours ---- */
  --bg: #fff3dc;          /* page background: warm cream */
  --ink: #3b2314;         /* text and borders: cocoa brown */
  --accent: #d9482b;      /* hover, highlight: chilli red */
  --card: #ffffff;        /* cards and header */
  --danger: #b02a1a;      /* clear / remove buttons */
  --leaf: #4c8c4a;        /* fresh green */
  --turmeric: #f2b33d;    /* warm yellow highlight */

  /* ---- Ticket and status colours ---- */
  --t-yellow: #ffe08a;    /* Received */
  --t-blue: #ffd0a8;      /* Preparing (peach) */
  --t-mint: #cfeab0;      /* Ready (fresh leaf) */

  /* ---- Soft tints ---- */
  --bg-soft: #fff8ec;
  --ink-soft: #7a5a44;
  --line: #f0d7b8;
  --column: rgba(255, 255, 255, .6);

  /* ---- Shape and shadow ---- */
  --radius: 16px;
  --shadow: 4px 4px 0 var(--ink);
  --shadow-small: 2px 2px 0 var(--ink);

  padding-top: env(safe-area-inset-top, 0px);
  padding-bottom: env(safe-area-inset-bottom, 0px);
}

* { box-sizing: border-box; }

/* Cream background with a faint food doodle pattern (burger, pizza, tea, samosa, fries, drumstick) */
body {
  margin: 0;
  background-color: var(--bg);
  background-image: url("data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' width='320'
      height='320' viewBox='0 0 320 320'><g fill='none' stroke='%233b2314' stroke-width='3'
      stroke-linecap='round' stroke-linejoin='round' opacity='0.10'><path d='M18 60 Q40 28 62 60
      Z'/><path d='M16 66 H64'/><path d='M18 72 Q40 80 62 72'/><path d='M16 78 H64'/><path d='M18 84 Q40
      93 62 84 Z'/><path d='M160 40 L192 104 L128 104 Z'/><circle cx='158' cy='72' r='3'/><circle
      cx='170' cy='86' r='3'/><circle cx='150' cy='92' r='3'/><path d='M238 62 H272 V82 Q272 98 255 98
      Q238 98 238 82 Z'/><path d='M272 68 Q284 68 284 78 Q284 86 272 86'/><path d='M247 52 Q244 45 248
      38'/><path d='M258 52 Q255 45 259 38'/><path d='M36 178 L68 146 L100 178 Z'/><path d='M52 168 L84
      168'/><path d='M146 192 L152 160 H184 L190 192 Z'/><path d='M158 160 V148'/><path d='M168 160
      V146'/><path d='M178 160 V149'/><path d='M244 206 Q234 196 242 184 Q254 176 266 182 Q280 188 272
      204 Q266 216 244 206 Z'/><path d='M266 180 L280 168'/><path d='M120 270 Q112 258 122 250 Q136 246
      146 258 Q148 270 136 274 Q126 276 120 270 Z'/><path d='M230 280 Q242 262 262 268 Q278 276 268 292
      Q254 302 230 280 Z'/></g></svg>");
  background-size: 320px 320px;
  color: var(--ink);
  font-family: "Trebuchet MS", "Segoe UI", system-ui, sans-serif;
  line-height: 1.5;
  overflow-x: hidden;
  animation: fadeIn .6s ease both;
}

h1, h2, h3 { margin: 0; line-height: 1.2; letter-spacing: -.3px; }
.muted { color: var(--ink-soft); }
.hidden { display: none !important; }

/* ---------- Buttons ---------- */
button {
  font: inherit;
  border: 2px solid var(--ink);
  border-radius: 12px;
  padding: 9px 16px;
  background: var(--accent);
  color: #ffffff;
  font-weight: 700;
  cursor: pointer;
  box-shadow: var(--shadow-small);
```

```
  transition: transform .15s, box-shadow .15s, background .15s, color .15s;
  white-space: nowrap;
}
button:hover {
  background: var(--ink);
  color: #ffffff;
  transform: translate(-2px, -2px);
  box-shadow: 4px 4px 0 var(--accent);
  animation: wiggle .5s ease;
}
button:active { transform: translate(2px, 2px); box-shadow: none; }
button:disabled {
  opacity: .5;
  cursor: not-allowed;
  transform: none;
  box-shadow: var(--shadow-small);
  background: var(--ink-soft);
  animation: none;
}
button.ghost { background: var(--card); color: var(--ink); }
button.ghost:hover { background: var(--bg-soft); color: var(--ink); box-shadow: 4px 4px 0
    var(--turmeric); }
button.small { padding: 5px 11px; font-size: 14px; }
button.danger { background: var(--card); color: var(--danger); border-color: var(--danger); }
button.danger:hover { background: var(--danger); color: #ffffff; box-shadow: 4px 4px 0 var(--ink); }
button:focus-visible, input:focus-visible, select:focus-visible, a:focus-visible {
  outline: 3px solid var(--turmeric);
  outline-offset: 2px;
}

/* Spinner: shown on any button that has the class "loading" */
button.loading { pointer-events: none; }
button.loading::after {
  content: "";
  display: inline-block;
  width: 14px; height: 14px;
  margin-left: 8px;
  vertical-align: -2px;
  border: 3px solid #ffffff;
  border-right-color: transparent;
  border-radius: 50%;
  animation: spin .7s linear infinite;
}

/* ---------- Inputs ---------- */
input, select {
  font: inherit;
  padding: 9px 12px;
  border: 2px solid var(--ink);
  border-radius: 12px;
  background: var(--card);
  color: var(--ink);
  max-width: 100%;
}
label { display: block; font-weight: 700; font-size: 14px; }
label.field { margin-bottom: 12px; }
label input, label.field input { display: block; width: 100%; margin-top: 4px; }

/* ---------- Header and tabs ---------- */
header.top {
  background: var(--card);
  border-bottom: 3px solid var(--ink);
  padding: 12px 24px;
  display: flex;
  align-items: center;
  gap: 16px;
  flex-wrap: wrap;
  position: sticky;
  top: 0;
  z-index: 5;
}
.brand { font-size: 24px; font-weight: 900; letter-spacing: -1px; color: var(--ink); }
.logo {
  display: inline-grid; place-items: center;
  width: 32px; height: 32px; margin-right: 8px; vertical-align: -7px;
  border-radius: 9px;
  background: var(--accent); color: var(--card);
  font-size: 19px; font-weight: 900; letter-spacing: 0;
```

```
  box-shadow: var(--shadow-small);
}
.tabs { display: flex; gap: 6px; flex-wrap: wrap; margin-left: 16px; }
.tab {
  background: transparent;
  color: var(--ink);
  border: 2px solid transparent;
  box-shadow: none;
  text-decoration: none;
  padding: 8px 14px;
  border-radius: 999px;
}
.tab:hover { background: var(--bg-soft); color: var(--accent); transform: none; box-shadow: none;
    animation: none; }
.tab.active {
  background: var(--turmeric);
  border-color: var(--ink);
  box-shadow: var(--shadow-small);
  color: var(--ink);
}
header.top .right { margin-left: auto; }

/* ---------- Layout ---------- */
main { max-width: 760px; margin: 0 auto; padding: 24px; }
main.wide { max-width: 1100px; }
main.narrow { max-width: 760px; margin: 0 auto; padding: 24px; }
.panel { display: none; }
.panel.active { display: block; animation: reveal .45s ease both; }
.view { display: none; }
.view.active { display: block; animation: reveal .45s ease both; }

/* ---------- Cards ---------- */
.card {
  background: var(--card);
  border: 2px solid var(--ink);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 20px;
  margin-bottom: 20px;
}
.card h2, .card h3 { font-size: 18px; margin-bottom: 12px; }
a.card { color: var(--ink); display: block; text-decoration: none; transition: transform .15s,
    box-shadow .15s; }
a.card:hover { transform: translate(-2px, -2px); box-shadow: 6px 6px 0 var(--accent); }
a.card h2 { color: var(--accent); margin-bottom: 6px; }
a.card p { margin: 0; color: var(--ink-soft); }
a.card .go { display: inline-block; margin-top: 12px; font-weight: 800; color: var(--ink); }
.grid { display: grid; grid-template-columns: repeat(2, 1fr); gap: 16px; }
.soon {
  margin-top: 16px;
  background: var(--turmeric);
  border: 2px dashed var(--ink);
  border-radius: var(--radius);
  padding: 18px 22px;
  color: var(--ink);
  font-weight: 600;
}

/* ---------- Hero (top banner) ---------- */
.hero { margin-bottom: 20px; }
.hero h1 { font-size: 30px; margin-bottom: 6px; color: var(--accent); }
.hero p { margin: 0; color: var(--ink-soft); }

/* ---------- Rush radar ---------- */
.rush { display: flex; justify-content: space-between; align-items: center; gap: 12px; }
.rush b { font-weight: 800; }
.rush .num, .rush #rush-num { font-size: 38px; font-weight: 900; color: var(--accent); }

/* ---------- Menu items: lift slightly on hover ---------- */
.item {
  display: flex; justify-content: space-between; align-items: center; gap: 12px;
  padding: 12px 10px;
  margin: 0 -10px;
  border-bottom: 2px dashed var(--line);
  border-radius: 12px;
  transition: transform .2s ease, background .2s ease, box-shadow .2s ease;
}
.item:last-child { border-bottom: 0; }
```

```
.item:hover {
  transform: translate(-2px, -2px);
  background: var(--bg-soft);
  box-shadow: 3px 3px 0 var(--turmeric);
}
.item > div:first-child { min-width: 0; flex: 1; }
.name { font-weight: 700; font-size: 16px; overflow-wrap: anywhere; }
.item .meta { color: var(--ink-soft); font-size: 14px; }
.badge {
  display: inline-block;
  padding: 2px 10px;
  border: 2px solid var(--ink);
  border-radius: 999px;
  font-size: 13px;
  font-weight: 700;
  background: var(--turmeric);
  color: var(--ink);
  white-space: nowrap;
}
.badge.green { background: var(--t-mint); }
.badge.off { background: var(--bg-soft); color: var(--ink-soft); }
.stepper { display: flex; align-items: center; gap: 10px; flex-shrink: 0; }
.stepper span {
  min-width: 24px; text-align: center; font-weight: 800;
  transition: transform .2s ease;
}
.price { font-weight: 900; white-space: nowrap; color: var(--accent); }
.off-text { color: var(--ink-soft); }
.empty { color: var(--ink-soft); padding: 8px 0; }

/* ---------- List rows (Running discounts, Done orders) ---------- */
.list-row {
  display: flex; justify-content: space-between; align-items: center;
  gap: 12px; flex-wrap: wrap;
  padding: 10px 0;
  border-bottom: 2px dashed var(--line);
}
.list-row:last-child { border-bottom: 0; }
.list-row > span { min-width: 0; overflow-wrap: anywhere; flex: 1 1 200px; }
.list-row button { flex-shrink: 0; }

/* ---------- Form row (inputs and button in one row) ---------- */
.form-row { display: flex; gap: 10px; flex-wrap: wrap; align-items: center; }
.form-row input, .form-row select { flex: 1 1 160px; min-width: 0; }
.form-row button { flex-shrink: 0; }

/* ---------- Cart bar (student): slides up when the page opens ---------- */
.cart {
  position: sticky; bottom: 12px;
  background: var(--ink); color: var(--card);
  border-radius: var(--radius);
  padding: 14px 18px;
  margin-bottom: 20px;
  display: flex; justify-content: space-between; align-items: center; gap: 12px;
  box-shadow: 4px 4px 0 var(--accent);
  animation: slideUp .6s cubic-bezier(.2, .9, .3, 1.2) both;
}
.cart button { background: var(--turmeric); color: var(--ink); border-color: var(--turmeric); }
.cart button:hover { background: var(--accent); color: #ffffff; border-color: var(--accent); }

/* ---------- Token tracking (student) ---------- */
.ticket-big { text-align: center; padding: 8px 0; }
.ticket-big small { color: var(--ink-soft); font-size: 14px; }
.ticket-big .tok {
  font-size: 56px; font-weight: 900; line-height: 1.1;
  color: var(--accent);
  display: inline-block;
}
/* Token pulses while the order is Ready (the third step is the current one) */
.card:has(.step:nth-child(3).now) .tok { animation: pulse 1.1s ease-in-out infinite; }

.steps { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; margin: 16px 0; }
.step {
  text-align: center; padding: 12px 6px; border-radius: 12px;
  border: 2px solid var(--ink);
  background: var(--card); color: var(--ink-soft);
  font-weight: 700; font-size: 14px;
  transition: background .4s ease, color .4s ease;
```

```
}
.step span { display: block; font-size: 20px; font-weight: 900; }
.step.done { background: var(--t-mint); color: var(--ink); }
/* The current step glows softly */
.step.now {
  background: var(--accent); color: #ffffff;
  box-shadow: var(--shadow-small);
  animation: glow 1.4s ease-in-out infinite;
}
.track-msg { text-align: center; font-weight: 700; margin-bottom: 12px; }

/* Token card fades in when it appears */
#track-card:not([hidden]) { animation: reveal .5s ease both; }

/* ---------- Forms and planner ---------- */
.form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }
.summary {
  display: flex; justify-content: space-between; gap: 12px; flex-wrap: wrap;
  background: var(--t-yellow);
  border: 2px solid var(--ink);
  border-radius: 12px;
  padding: 12px 16px;
  margin-bottom: 12px;
}
.summary b { font-size: 18px; }
.error { color: var(--danger); margin: 8px 0 0; min-height: 22px; font-weight: 700; }

/* ---------- Staff: stats ---------- */
.stats { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px; margin-bottom: 14px; }
.stat {
  background: var(--card);
  border: 2px solid var(--ink);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 16px;
}
.stat .muted { font-size: 15px; }
.stat .num { font-size: 34px; font-weight: 900; color: var(--ink); }
.stat.hot .num { color: var(--accent); }
.staff-actions { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 16px; }

/* ---------- Staff: order board ---------- */
.board { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; align-items: start;
    margin-bottom: 20px; }
.col {
  background: var(--column);
  border: 2px dashed var(--ink);
  border-radius: var(--radius);
  padding: 12px;
  min-height: 150px;
}
.col h3 { font-size: 16px; margin-bottom: 10px; display: flex; justify-content: space-between;
    align-items: center; }
.count {
  background: var(--accent); color: var(--card);
  border-radius: 999px; padding: 0 10px; font-size: 14px; font-weight: 700;
}

.ticket {
  background: var(--card);
  border: 2px solid var(--ink);
  border-radius: 12px;
  margin-bottom: 10px;
  display: flex;
  overflow: hidden;
  box-shadow: var(--shadow-small);
}
.stub {
  width: 78px; flex-shrink: 0;
  display: flex; flex-direction: column;
  align-items: center; justify-content: center;
  padding: 10px 4px;
  border-right: 2px dashed var(--ink);
}
.stub small { font-size: 12px; color: var(--ink-soft); }
.stub b { font-size: 26px; font-weight: 900; color: var(--ink); }

/* Status colours: Received = turmeric, Preparing = peach, Ready = fresh leaf */
```

```
.ticket.Received .stub, .ticket.Pending .stub, .ticket.t-r .stub { background: var(--t-yellow); }
.ticket.Preparing .stub, .ticket.t-p .stub { background: var(--t-blue); }
.ticket.Ready .stub, .ticket.t-d .stub { background: var(--t-mint); }

.tb, .ticket .body { padding: 10px 12px; flex: 1; min-width: 0; }
.tb .it, .ticket .items { font-weight: 700; word-break: break-word; }
.tb .mt, .ticket .meta { font-size: 13px; color: var(--ink-soft); margin: 4px 0 8px; }
.ticket button { margin-top: 2px; }

/* ---------- Staff: menu table ---------- */
.table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; }
th, td { text-align: left; padding: 10px 8px; border-bottom: 2px dashed var(--line); vertical-align:
    middle; }
th { font-size: 14px; color: var(--ink-soft); font-weight: 700; }
td.actions { display: flex; flex-wrap: wrap; gap: 6px; }

/* ---------- Login (staff) ---------- */
#login-screen { min-height: 100vh; display: grid; grid-template-columns: 1fr 1fr; }
.login-brand {
  background: var(--accent);
  color: var(--card);
  padding: 48px;
  display: flex; flex-direction: column; justify-content: center; gap: 16px;
  border-right: 3px solid var(--ink);
}
.login-brand .logo {
  display: inline-block;
  width: auto; height: auto;
  padding: 10px 20px;
  font-size: 48px; font-weight: 900; line-height: 1.1; letter-spacing: -1px;
  background: var(--turmeric);
  color: var(--ink);
  border-radius: 14px;
  box-shadow: 4px 4px 0 var(--ink);
  margin: 0;
}
.login-brand p { margin: 0; font-size: 18px; max-width: 32ch; color: var(--card); }
.login-form { display: flex; align-items: center; justify-content: center; padding: 32px; background:
    var(--bg); }
.login-form form { width: 100%; max-width: 340px; }
.login-form h2 { font-size: 26px; margin-bottom: 4px; color: var(--accent); }
.login-form .msg { min-height: 24px; color: var(--danger); margin: 8px 0; font-weight: 700; }
.login-form button { width: 100%; padding: 12px; }

/* ---------- Toast ---------- */
#toast {
  position: fixed; bottom: 80px; left: 50%; transform: translateX(-50%);
  background: var(--ink); color: var(--card);
  padding: 10px 18px; border: 2px solid var(--ink);
  border-radius: 12px; z-index: 10; font-weight: 700;
  max-width: calc(100vw - 32px); text-align: center;
  animation: reveal .3s ease both;
}

/* ---------- Keyframes ---------- */
@keyframes wiggle {
  0%, 100% { transform: translate(-2px, -2px) rotate(0); }
  25% { transform: translate(-2px, -2px) rotate(-2deg); }
  75% { transform: translate(-2px, -2px) rotate(2deg); }
}
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.12); }
}
@keyframes glow {
  0%, 100% { box-shadow: 0 0 0 0 rgba(217, 72, 43, .45); }
  50% { box-shadow: 0 0 0 7px rgba(217, 72, 43, 0); }
}
@keyframes spin { to { transform: rotate(360deg); } }
@keyframes reveal {
  from { opacity: 0; transform: translateY(12px); }
  to { opacity: 1; transform: none; }
}
@keyframes slideUp {
  from { opacity: 0; transform: translateY(140%); }
  to { opacity: 1; transform: translateY(0); }
}
```

```
@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }

/* ---------- Responsive (phones) ---------- */
@media (max-width: 700px) {
  main, main.narrow { padding: 16px; }
  header.top { padding: 12px 16px; gap: 10px; }
  .tabs {
    margin-left: 0; width: 100%;
    flex-wrap: nowrap; overflow-x: auto;
    -webkit-overflow-scrolling: touch;
    padding-bottom: 4px;
  }
  .tab { flex-shrink: 0; white-space: nowrap; }

  .form-grid, .stats, .board, .grid { grid-template-columns: 1fr; }
  .hero h1 { font-size: 24px; }
  .steps { gap: 6px; }

  .item { flex-wrap: wrap; }
  .item > div:first-child { flex: 1 1 60%; }
  .stepper { margin-left: auto; }

  .stub { width: 66px; }
  .stub b { font-size: 22px; }
  .tb, .ticket .body { padding: 10px; }

  .list-row { flex-direction: column; align-items: flex-start; }
  .list-row button { width: auto; }

  .form-row input, .form-row select, .form-row button { flex: 1 1 100%; width: 100%; }

  .table-wrap { overflow-x: visible; }
  table, thead, tbody, th, td, tr { display: block; width: 100%; }
  thead { display: none; }
  tr {
    border: 2px solid var(--ink);
    border-radius: 12px;
    padding: 10px;
    margin-bottom: 12px;
    background: var(--card);
    box-shadow: var(--shadow-small);
  }
  td { border: none; padding: 4px 0; }
  td.actions { margin-top: 6px; }

  #login-screen { grid-template-columns: 1fr; }
  .login-brand { padding: 32px 24px; }
}

/* Users who prefer less motion get no animation */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation: none !important; transition: none !important; }
}
```

B.7 README.md

```
# BreakBite

A smart canteen web project for the Gono Bishwabidyalay CSE Software Development Contest 2026 (Category
    1: Semester 1st to 4th).

## About

BreakBite helps students spend less time waiting in the university canteen. Students can see the menu,
    order ahead, get a token and know the waiting time. Staff can manage orders and food from their own
    page.

## Features

- Digital Menu
- Token System
- Waiting Time
- Pre-order
- Rush Radar
- Break Planner
- Green Token
- Food Waste Saver

## Team

| Name | Student ID | Responsibility |
|---|---|---|
| Nahar Mim (Team Lead) | 26251037 | Lead and documentation |
| Sadnan Ahmed | 26251038 | Student pages |
| Wasika Jahan | 26251026 | UI design |
| Mirza Muin Labbaik | 26251021 | Staff page and database |
| Habiba Jannat Taiyaba | 26251011 | Break Planner and testing |

## Folder Structure

- `/index.html` : home page (Mim)
- `/student.html` : student pages (Sadnan)
- `/staff.html` : staff page and database (Labbaik)
- `/planner.html` : Break Planner and testing (Taiyaba)
- `/style.css` : UI design and styling (Wasika)
- `README.md` : project information (Mim)

## How to Work Together

1. Open GitHub Desktop and click Fetch origin, then Pull origin before starting work.
2. Work only on your own files so changes do not clash.
3. Click Commit with a short message about what you changed.
4. Click Push origin.

## Important Dates

- Final project submission: 12 November 2026
- Showcase and poster: 16 November 2026
```

B.8 .gitattributes

```
# Auto detect text files and perform LF normalization
* text=auto
```